

Distributed Flow Control for Efficient DNN Training Scheduling

Heng Xu¹, Chengze Du¹, Zhiwei Yu², Letian Li³, Ying Zhou¹, Bo Liu¹, Jialong Li^{1,✉}

¹ Faculty of Computer Science and Artificial Intelligence, Shenzhen University of Advanced Technology

² Institute for Network Sciences and Cyberspace, Tsinghua University

³ Department of Information Engineering, Chinese University of Hong Kong

✉lijialong@suat-sz.edu.cn

Abstract—Distributed Deep Neural Network (DNN) training generates periodic, long-lived, and interdependent flows that contrast sharply with the short, bursty, and independent flows typical of traditional cloud services. Existing flow scheduling methods, optimized for cloud traffic, struggle to handle the structured communication of DNN workloads, while static schedulers remain brittle under the computation jitter and stochasticity inherent in multi-tenant AI clusters. We propose a distributed traffic control and scheduling framework called PQ, which shifts from fragile global synchronization to a token-based queuing concept. PQ utilizes standard priority queues in commercial switches as elastic buffers, dynamically mapping task urgency to traffic priorities based on specific scheduling policies, such as minimizing waiting time, thereby accelerating efficiency. Results show that PQ achieves stable communication interleaving $3.6\times$ to $8.8\times$ faster than reactive baselines like MLTCP and FQ. Furthermore, it significantly optimizes performance by reducing average iteration time by up to 29.2% while maintaining higher link utilization.

Index Terms—Distributed DNN training, priority queue, flow scheduling

I. INTRODUCTION

Deep Neural Network (DNN) training has evolved into a dominant workload in modern data centers [21], [18], [47], [53]. As model sizes and cluster scales grow, network performance becomes the bottleneck governing both training efficiency and cost. While flow control and scheduling are fundamental for resource allocation, existing solutions are largely tailored for traditional cloud services characterized by short, bursty, and independent flows [44], [57], [32], [25]. In stark contrast, DNN training traffic is periodic, bandwidth-intensive, and exhibits strong inter-flow dependencies dictated by the iterative nature of training algorithms [26], [33], [40], [36]. These distinctions render conventional cloud scheduling policies ineffective for AI workloads.

The transition from dedicated clusters to multi-tenant AI clouds introduces critical stochasticity where multiple users concurrently train diverse models. Unlike the idealized assumptions in prior works, the computation time of training iterations fluctuates due to hardware thermal throttling and resource contention in pooled or virtualized environments. This unpredictability is fatal for centralized and static schedulers like Cassini [43]. These approaches [58], [1], [39] rely on pre-calculated time shifts based on perfect periodicity to interleave communication. However, even minor computation jitter in

a dynamic multi-tenant setting can invalidate these static schedules and cause traffic collisions. Furthermore, forcing expensive GPUs to idle while waiting for synchronized time slots is economically inefficient. Thus, there is an urgent need for a scheduling mechanism that is robust against timing variations and capable of achieving communication interleaving without assuming a static world.

On one hand, traditional distributed schedulers [55], [9], [8], [20], [2] largely lack application awareness, which often leads to suboptimal performance. They are optimized for flow-level fairness or minimizing the flow completion time of random, independent traffic. However, these schemes are oblivious to the rigid dependency chains in DNN training, where specific communication flows directly block the next computation phase. By treating critical blocking flows with the same egalitarian fairness as non-critical background traffic, these policies inadvertently stall the entire training pipeline, leading to significant GPU idling and prolonged iteration times.

On the other hand, recent DNN-specific solutions, most notably MLTCP [42], attempt to approximate communication interleaving through reactive end-host rate adaptation. These methods introduce a specialized congestion control logic that adjusts the congestion window based on the remaining bytes of a flow to create transmission gaps. However, this approach faces two fundamental hurdles. First, the iterative process of probing for the correct rate based on remaining bytes is inherently slow to converge. In the microsecond-scale environment of AI clusters, this slow reaction time makes it impossible to adapt to transient computation jitter. As illustrated in Fig. 1, this leads to persistent traffic oscillation and collisions under high load intensities, where the reactive logic fails to stabilize the interleaving window. Second, MLTCP and its derivatives often necessitate non-standard modifications to the underlying network hardware, such as specialized switch buffer management or custom NIC logic. This significantly hinders their practical deployment in large-scale, commodity data centers. Ultimately, these rate-based methods offer a fragile software-level approximation that lacks the agility and hardware compatibility required for dynamic, multi-tenant AI workloads.

In this paper, we propose a distributed priority-queue-based flow control and scheduling framework, denoted as PQ, for robust communication interleaving in multi-tenant AI clusters.

Instead of fragile global synchronization or slow reactive rate control, PQ adopts a tag and queue philosophy leveraging the standard priority queuing mechanisms available in commodity switches. The key insight is that priority queues naturally act as robust buffers against stochasticity. If traffic arrives irregularly due to computation jitter, priority scheduling ensures that critical flows blocking computation are processed immediately while non-critical flows are buffered. This automatically achieves interleaving without requiring precise global clocks. PQ achieves intelligent control by dynamically mapping the non-communication to communication relationship of tasks to traffic priorities such as DSCP tags. We further consider two variants of PQ: PQ-T, a latency-oriented variant that minimizes normalized waiting time, and PQ-L, a throughput-oriented variant that maximizes link utilization.

To evaluate PQ, we developed a custom discrete-time simulator and conducted large-scale experiments on a 192-host hybrid topology with a 3:1 oversubscription ratio. The results reveal that PQ significantly outperforms existing solutions: it reaches stable communication interleaving $3.6\times$ to $8.8\times$ faster than MLTCP and FQ, respectively. Notably, PQ-L achieves near-optimal link utilization. Under aggressive contention (intensity up to 1.0), PQ-T curtails queuing delays by $6.39\times$, effectively isolating critical flows through its priority-mapping mechanism. Unlike reactive rate-control methods that degrade under heavy load, PQ’s proactive and stateless design ensures predictable performance, making it a robust backbone for dynamic, multi-tenant AI workloads.

The contribution of PQ lies in two aspects. First, from a theoretical perspective, we show that for the objective of minimizing normalized waiting time, the globally optimal scheduling order can be determined solely from locally measurable task characteristics, namely the non-communication duration and communication time, without requiring centralized global-state coordination. Second, from a system perspective, we realize these derived rules as a fully distributed cross-layer framework, where each host independently extracts local metrics, computes priority scores, maps them to DSCP values through deterministic quantization, and enforces them through the joint operation of commodity-switch priority queues and transport-layer collaborative control. This makes PQ fundamentally different from prior priority-based approaches that either rely on centralized coordination, assume generic flow-level priorities, or do not expose the connection between local observability and objective-driven optimal scheduling.

II. MOTIVATION

In this section, we analyze the limitations of existing static scheduling approaches in production AI clusters, including sensitivity to computation jitter (§II-A), poor scalability under multi-tenancy (§II-B), and inefficient GPU idling (§II-C). We then motivate priority queues as a robust and work-conserving alternative for handling stochastic traffic contention (§II-D).

A. Unpredictable Computation Times

Existing network-aware schedulers assume perfect periodicity. They model each training iteration as a deterministic

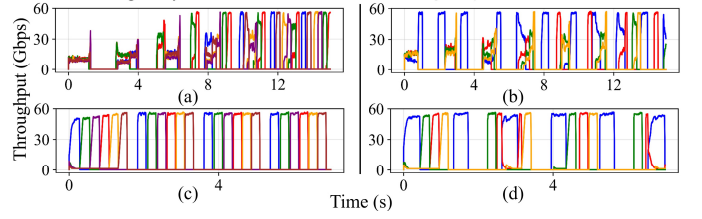


Fig. 1: Temporal evolution of communication interleaving under MLTCP and PQ in homogeneous and heterogeneous workloads. (a) and (b) show that MLTCP exhibits persistent oscillation and traffic collisions, especially under workload heterogeneity. In contrast, (c) and (d) show that PQ quickly converges to a stable interleaving pattern and remains robust under both homogeneous and heterogeneous settings.

cycle with fixed computation and communication phases, and then pre-calculate time offsets to interleave communication windows. This assumption is often violated in production environments, where computation duration is a stochastic variable rather than a constant. Hardware effects such as thermal throttling and error correction introduce timing variation, and operating-system jitter further amplifies it. For a static scheduler, even a small delay can shift the next communication phase into another job’s reserved window, causing unexpected collisions and degraded network performance. Recomputing a global schedule for every such perturbation is computationally prohibitive. Therefore, static scheduling is inherently brittle in stochastic environments.

B. Complexity of Multi-Tenancy

The trend towards multi-tenant AI clusters further exacerbates scheduling challenges. In virtualized environments, resources are pooled and a training job often shares physical infrastructure with noisy neighbors. This interference introduces non-stationary noise into the execution timeline. Furthermore, static schedulers often rely on calculating the least common multiple of iteration times to construct a global schedule. In a cluster with diverse models, this common cycle grows exponentially large and makes the scheduling horizon intractable. The dynamic arrival and departure of tenants require constant re-computation of these global schedules. This creates a synchronization bottleneck that cannot scale effectively.

C. High Cost of GPU Idling

A critical drawback of time-shifting schedulers is their economic inefficiency. Mechanisms that enforce interleaving by delaying the start of a job are non-work-conserving. They intentionally force computation units to be idle to ensure clear network paths. Given that high-end accelerators are the most expensive resource in the data center, optimizing for network jitter at the expense of utilization is economically counter-intuitive. A viable solution must be work-conserving and utilize network buffers to absorb contention rather than halting expensive compute resources.

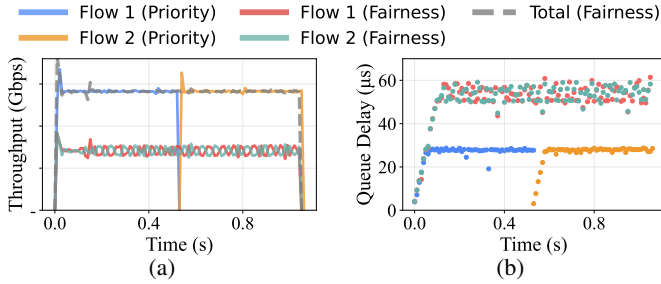


Fig. 2: Comparison between fairness-based and priority-based scheduling at a bottleneck link. (a) Throughput traces show that priority scheduling lets critical traffic complete earlier while non-critical traffic temporarily yields, reducing overlap. (b) Queue-delay traces show that priority scheduling keeps the delay of critical traffic low, whereas fairness scheduling leads to persistently higher queueing delay under contention.

D. Priority Queues as Robust Buffers

To address these challenges, we argue for a shift from prediction-based scheduling to priority-based buffering. We observe that while exact timing is unpredictable, the relative urgency of traffic is definable. Priority queues offer a natural solution as they act as elastic buffers. When traffic bursts collide due to jitter, priority queues enforce interleaving by processing critical data immediately while buffering non-critical data. This approach absorbs stochasticity without requiring global clock synchronization or resource idling and makes it the ideal primitive for multi-tenant AI clusters.

As illustrated in Fig. 2, traditional fairness-based scheduling often leads to suboptimal performance due to persistent resource contention. By shifting to a priority-based strategy, we prioritize the urgency of tasks over flow-level egalitarianism. While this may introduce short-term unfairness at the flow level, it significantly optimizes throughput and reduces queue delay for the overall training task, which is a long-term complex process rather than a collection of isolated flows.

III. RELATED WORK

Flow Control and Congestion Control. Congestion control algorithms generally fall into three categories: drop-based [19], [54], [12], [14], delay-based [6], [24], [5], and ECN-based [3], [60]. Early protocols such as TCP Reno [19] relied on packet loss as the primary congestion signal, while later designs improved adaptability by exploiting delay signals [6] or explicit congestion notifications [3], with the latter being widely adopted in datacenter environments [60]. To address increasingly diverse workload requirements, more recent designs further introduced deadline-awareness [51], in-network scheduling [23], [11], [48], and receiver-driven transport mechanisms [15], [34]. These advances have significantly improved the efficiency and responsiveness of generic datacenter traffic management. However, these approaches focus on generic datacenter traffic and fall short in addressing the specific scheduling and communication demands.

Traditional Data Center Scheduling. Early datacenter scheduling primarily aimed to minimize flow completion time

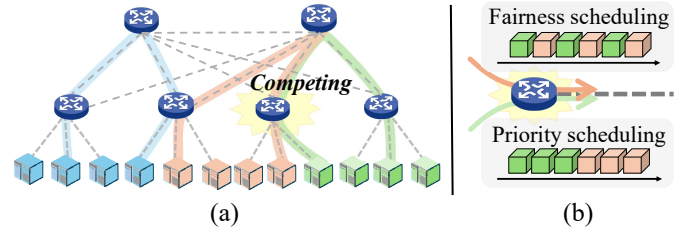


Fig. 3: (a) DNN training Competing and (b) task scheduling mode.

and improve link utilization by targeting independent and short-lived flows. Centralized methods such as Hedera [1] and FastPass [39] can achieve near-optimal scheduling decisions, but their reliance on global state limits scalability and responsiveness in dynamic environments. Coflow-based schedulers [9], [8], [20] better capture dependencies among related flows and improve coordination at the job level, yet they still pay limited attention to inter-job contention on shared links. Protocols such as pFabric [2] further optimize flow completion time through priority-based scheduling, but they are built on strong assumptions about workload characteristics and network behavior, which restrict their applicability in stochastic and multi-tenant settings. As a result, these designs are less effective for modern AI clusters, where traffic is highly structured, long-lived, and tightly coupled with iterative training progress.

DNN Schedulers. The iterative, communication-intensive nature of DNN training renders traditional datacenter schedulers inadequate [35], [30]. Early work focused on compute resource allocation [52], [28], [40], [13], [59], [37], [21], often addressing network issues only indirectly via coarse placement. Later methods improve intra-job communication using collective algorithms [45], [22], pipelining [16], or tensor-level scheduling [38], [29], but largely overlook inter-job contention on shared links. D-Credit [50] proposes a dynamic sliding-window tensor communication framework that adaptively controls the number of concurrently transmitted tensor blocks to improve communication-computation overlap and reduce per-iteration time. FlowMoE [49] extends this line to distributed MoE training by constructing a unified pipeline over MHA, gating, expert computation, A2A communication, and all-reduce, and further introduces tensor chunk-based priority scheduling to overlap all-reduce with computation. Muri [58] tackles this through interleaving in GPU-sharing, CASSINI [43] aligns job phases via geometric and affinity models, and MLTCP [42] enables decentralized coordination via short-term unfairness. These works indicate that strict bandwidth fairness is often suboptimal, consistent with controlled unfairness studies [56], [4] and our own results.

IV. SYSTEM MODELING AND SCHEDULING STRATEGY

In this section, we abstract the traffic characteristics of distributed DNN training (§IV-A) and derive optimal flow scheduling strategies. We formulate priority rules for maximizing average link utilization (§IV-B) and minimizing nor-

malized waiting time (§IV-C), and finally show how these rules can be enforced using standard priority queues (§IV-D).

A. Traffic Abstraction

A distributed training workload exhibits a hierarchical structure. A training job consists of thousands of iterations which repeat the same model execution logic. Crucially, a single iteration comprises a sequence of multiple alternating non-communication and communication phases. These phases exhibit varying durations and create a dynamic resource demand pattern.

For clarity, we use three core timing notations throughout this section: G denotes the effective local non-communication duration before a flow becomes ready, T denotes the transmission time of that flow on the bottleneck link, and D denotes the queuing delay before the flow obtains service on that link.

To enable a tractable analysis for network scheduling, we abstract the lifecycle of a training job around a bottleneck link into two alternating stages. The first is the non-communication stage, denoted by G , which represents the effective local execution time (e.g., computation or memory access) during which the job does not inject traffic into the bottleneck link. The second is the communication stage, denoted by T , which represents the time required to transmit the ready data volume L through the bottleneck link with capacity C , i.e., $T = L/C$. Accordingly, we use the pair (G, T) as the atomic scheduling unit, and later use D to denote the additional queuing delay before this communication stage can start.

Based on this, we define the atomic scheduling unit as the pair containing the completed non-communication duration G and the subsequent communication demand T . These inputs are measurable at the end-host, allowing our scheduler to make decisions based on the actual dynamic state of the training process. In other words, we are not scheduling complex dependent tasks, but rather focusing on a portion of each individual iteration. See Table I for the symbol.

B. Maximization of Average Link Utilization

In modern production AI clusters, the high-bandwidth gradient synchronization of massive DNN models leads to intense traffic convergence at the core switches. As multiple training jobs concurrently inject traffic into the network, the available buffer space rapidly depletes, leading to a backlog in the switch queue. At this critical juncture, as depicted in Fig. 3, the system could invoke a scheduling policy to resolve the contention. Different scheduling decisions yield vastly different performance outcomes; an arbitrary or purely fair allocation often results in overlapping communication windows that delay the computation start for all competing jobs.

To address this problem, our design utilizes a hardware priority queue to differentiate the transmission behavior of competing flows. Figure 4 shows the dynamic changes at the bottleneck link. By assigning different priority tags, the priority queue ensures that different tasks are served at different transmission rates. Specifically, high-priority flows

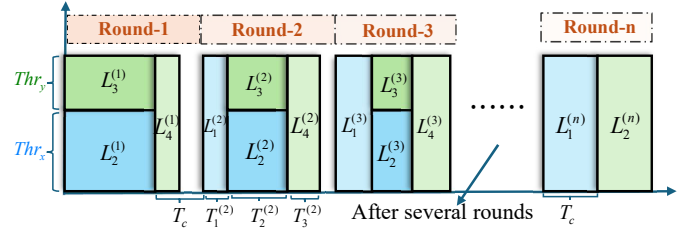


Fig. 4: The convergence of flows leads to exclusive link.

are given the majority of the link bandwidth. In contrast, the effective rate of low-priority flows is limited. This rate-based differentiation naturally leads to traffic staggering. High-priority tasks complete their communication ahead of time, while low-priority traffic yields the path. This autonomous interleaving quickly restores the exclusive link state, perfectly matching the ideal conflict-free scheduling shown in Fig. 2.

The core optimization problem is to determine an ordering of competing jobs that improves overall system performance. Consider a set of N independent jobs, where each job i is characterized by a non-communication interval G_i and a transmission interval T_i . We assume that access to the bottleneck link is sequential, while the non-communication phases of queued jobs may proceed in parallel with earlier transmissions. Under this model, job i can start transmission only after all earlier jobs have cleared the link. Therefore, its queuing delay D_i is the sum of the transmission times of the jobs scheduled ahead of it.

Our primary objective is to derive a sorting criterion that maximizes the average link utilization. We define the link utilization contribution of an individual job as the ratio of its transmission time to its total turnaround time, which includes the queuing delay. The optimization problem is formally stated as maximizing the following objective function:

$$\max \frac{1}{N} \sum_{i=1}^N \frac{T_i}{G_i + T_i + D_i} \quad (1)$$

To determine the optimal ordering rule, we analyze the impact on total system link utilization by comparing the relative scheduling order of two arbitrary jobs i and j residing consecutively in the queue. Let D denote the cumulative transmission time of all jobs preceding this pair. Crucially, this base delay D is determined solely by the earlier jobs in the queue and remains invariant regardless of the relative ordering of i and j .

First, we evaluate the link utilization contribution when job i is scheduled immediately before job j . In this sequence, job i inherits the delay D , completing at $G_i + T_i + D$. Job j follows immediately, inheriting a delay of $D + T_i$, and completes at $G_j + T_j + D + T_i$. The combined link utilization contribution for this pair is:

$$E_{i \rightarrow j} = \frac{T_i}{G_i + T_i + D} + \frac{T_j}{G_j + T_j + D + T_i} \quad (2)$$

Conversely, consider the swapped order, where job j precedes job i . Here, job j is subject to the base delay D , while

job i is delayed by $D + T_j$. The resulting combined link utilization contribution is:

$$E_{j \rightarrow i} = \frac{T_j}{G_j + T_j + D} + \frac{T_i}{G_i + T_i + D + T_j} \quad (3)$$

The optimal sequence is determined by the sign of the difference $\Delta = E_{i \rightarrow j} - E_{j \rightarrow i}$. A positive Δ indicates that prioritizing i over j yields higher link utilization. To facilitate analysis, we rearrange the difference by grouping terms associated with each job:

$$\Delta = \left(\frac{T_i}{G_i + T_i + D} - \frac{T_i}{G_i + T_i + D + T_j} \right) - \left(\frac{T_j}{G_j + T_j + D} - \frac{T_j}{G_j + T_j + D + T_i} \right) \quad (4)$$

We apply the algebraic identity $\frac{1}{X} - \frac{1}{X+Y} = \frac{Y}{X(X+Y)}$ to simplify the grouped fractions. This transformation allows us to merge terms with distinct denominators.

For the first group (numerator T_i), we set $X = G_i + T_i + D$ and increment $Y = T_j$. For the second group (numerator T_j), we set $X = G_j + T_j + D$ and increment $Y = T_i$.

Substituting these back simplifies the expression to:

$$\Delta = \frac{T_i T_j}{(G_i + T_i + D)(G_i + T_i + D + T_j)} - \frac{T_j T_i}{(G_j + T_j + D)(G_j + T_j + D + T_i)} \quad (5)$$

Inspection reveals that the numerators of both resulting fractions are identical and positive ($T_i T_j$). Consequently, for Δ to be positive (favoring the order $i \rightarrow j$), the fraction corresponding to the first term must exceed the second. Mathematically, this requires the denominator of the first term to be strictly smaller than that of the second.

This leads to the necessary and sufficient condition for the optimal scheduling order. Job i should precede job j if and only if:

$$(G_i + T_i + D)(G_i + T_i + D + T_j) < (G_j + T_j + D)(G_j + T_j + D + T_i) \quad (6)$$

This inequality provides the dynamic sorting criterion for the system. Under the single-bottleneck abstraction above, ensuring this condition for every adjacent pair yields the optimal ordering for maximizing the average link utilization.

The above derivation is exact under a single effective bottleneck with sequential service and deterministic nominal transmission intervals. If multiple links become bottlenecks simultaneously, the waiting process can no longer be summarized by a single scalar delay D , and the rule should be interpreted as a local ordering principle at a dominant bottleneck rather than a globally exact result. Likewise, when congestion causes losses and retransmissions, the effective service time becomes stochastic, so the closed-form rule based on $T_i = L_i/C$ is no longer exact.

TABLE I: Summary of main notations and parameters.

Symbol	Meaning
G	Effective non-communication duration before a flow becomes ready
T	Transmission time of a flow on the bottleneck link, with $T = L/C$
D_i	Queuing delay of job/flow i before accessing the bottleneck link
L	Data volume to be transmitted
C	Bottleneck-link capacity
N	Number of competing jobs in the analytical model
S_i^T	PQ-T priority score, defined as $T_i(G_i + T_i)$
S_i^L	PQ-L priority score, defined as $T_i/(G_i + T_i)$

C. Minimization of Normalized Waiting Time

Complementing the link utilization maximization, a secondary yet critical objective is the minimization of normalized waiting time. This metric addresses system fairness by scaling the queuing delay of each job relative to its total execution cycle (the sum of non-communication and transmission durations). We model a system of N jobs, again characterized by G_i and T_i . Given that jobs must wait for all predecessors to clear the channel, the queuing time D_i is the sum of preceding transmission times. The optimization goal is to minimize the aggregate normalized waiting time across all jobs:

$$\min \sum \frac{D_i}{G_i + T_i} \quad (7)$$

To derive the optimal rule, we compare the aggregate system cost under two alternative scheduling sequences for a pair of jobs i and j . Let D represent the invariant cumulative transmission time of all jobs preceding a specific adjacent pair i and j .

Consider the case where job i is scheduled immediately before job j . Job i starts after delay D , contributing a normalized cost of $\frac{D}{G_i + T_i}$. Job j waits for an additional T_i , resulting in a cost of $\frac{D + T_i}{G_j + T_j}$. The local total cost for this sequence is:

$$C_{i \rightarrow j} = \frac{D}{G_i + T_i} + \frac{D + T_i}{G_j + T_j} \quad (8)$$

Alternatively, consider the swapped order where job j precedes job i . Job j now incurs only the base delay D (cost $\frac{D}{G_j + T_j}$), while job i is delayed by $D + T_j$ (cost $\frac{D + T_j}{G_i + T_i}$). The swapped local cost is:

$$C_{j \rightarrow i} = \frac{D}{G_j + T_j} + \frac{D + T_j}{G_i + T_i} \quad (9)$$

We compare the costs by calculating the difference $\Delta = C_{i \rightarrow j} - C_{j \rightarrow i}$. Substituting the cost functions yields:

$$\Delta = \left(\frac{D}{G_i + T_i} + \frac{D + T_i}{G_j + T_j} \right) - \left(\frac{D}{G_j + T_j} + \frac{D + T_j}{G_i + T_i} \right) \quad (10)$$

To isolate the effect of the base delay, we regroup terms by their denominators. The first group collects terms with $G_i + T_i$, and the second collects those with $G_j + T_j$:

$$\Delta = \left(\frac{D}{G_i + T_i} - \frac{D + T_j}{G_i + T_i} \right) + \left(\frac{D + T_i}{G_j + T_j} - \frac{D}{G_j + T_j} \right) \quad (11)$$

Simplifying within the groups reveals a crucial property: the base waiting time D cancels out completely. The first group

simplifies to $\frac{-T_j}{G_i+T_i}$, and the second to $\frac{T_i}{G_j+T_j}$. The difference equation reduces to:

$$\Delta = \frac{-T_j}{G_i+T_i} + \frac{T_i}{G_j+T_j} = \frac{T_i}{G_j+T_j} - \frac{T_j}{G_i+T_i} \quad (12)$$

For the sequence "i precedes j" to be optimal, we require $\Delta < 0$. This implies:

$$\frac{T_i}{G_j+T_j} < \frac{T_j}{G_i+T_i} \quad (13)$$

Since all time parameters are positive, we cross-multiply to obtain the final sorting condition:

$$T_i(G_i+T_i) < T_j(G_j+T_j) \quad (14)$$

This result indicates that minimizing normalized waiting time requires prioritizing jobs based on the product of their transmission time and their total cycle length. Specifically, the system achieves global optimality by scheduling jobs in ascending order of the value $T_i(G_i+T_i)$.

D. Enforcement via Priority Queues

While our scheduling logic operates at the granularity of the atomic scheduling unit, the enforcement is performed at the packet level. The end-host calculates the priority based on the derived optimal strategies and tags the packets with the corresponding DSCP value. The switches then strictly enforce this order via standard priority queues. This decoupling allows complex and dynamic scheduling policies to be deployed on commodity hardware without requiring per-packet logic in the network core.

E. Analytical Scope and Boundary Conditions.

The derivations in this section are developed under a single effective bottleneck abstraction. Specifically, we consider the case where the competing communication units contend for one dominant congested resource, and service at that resource is sequential, with the nominal transmission time modeled as $T = L/C$. Under this abstraction, the scalar waiting term D fully captures the impact of preceding transmissions on the current job. In a general multi-hop data center network with ECMP, however, different flows may traverse partially different paths and may interact at multiple congested points simultaneously. In such cases, D can no longer be interpreted as a single end-to-end scalar for all flows, and the closed-form ordering rules derived here should be understood as exact results for the dominant-bottleneck model, rather than a universal end-to-end optimum for arbitrary multi-hop topologies.

V. DESIGN

This section describes the implementation of PQ. Unlike centralized schedulers that rely on global coordination, PQ operates in a fully distributed manner with an autonomous agent running on each training server. We first describe how PQ achieves local traffic awareness (§V-A), then explain the

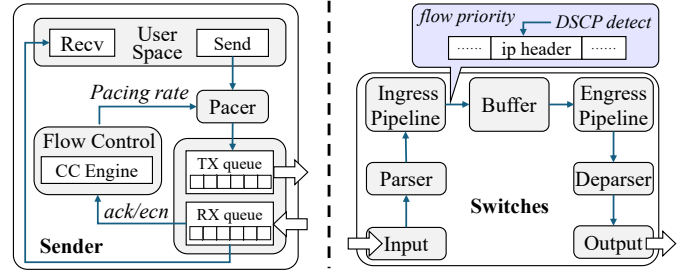


Fig. 5: Architecture of the PQ and its collaborative execution mechanism across the network and transport layers.

priority calculation and queue mapping mechanisms (§V-B), and finally present the collaborative execution across the network and transport layers (§V-C). The overall implementation logic is formalized in Algorithm 1.

A. Local Traffic Awareness

This design aims to solve a specific engineering challenge. Synchronizing the state information of all competing streams in a distributed network in real time is not only computationally expensive but also extremely complex, while relying on a central controller to collect and distribute global information introduces delays that compromise scheduling accuracy. Therefore, the PQ algorithm abandons reliance on global state information. It uses only locally observable metrics to execute the derived logic. The system performs a cyclical process: locally measuring task timestamps, calculating priority scores based on algebraic rules, and enforcing the ordering through hardware queues and transmission rate control.

The prerequisite for autonomy is self-awareness. As depicted in Fig. 5, the PQ agent is embedded in each training server and does not rely on a central controller to analyze tasks. Instead, it independently extracts atomic scheduling metrics G and T by intercepting the training loop, as formalized in Lines 2–5 of Algorithm 1.

The agent employs a non-intrusive timestamping mechanism. By hooking into communication primitives, it records the completion timestamp of the previous transmission t_{prev} and the start timestamp of the current transmission t_{curr} . The time interval $\Delta t = t_{curr} - t_{prev}$ represents the computation duration G . Given the cyclic nature of the training loop, we apply a sliding window average to the samples from the same execution stage across the last three iterations. This smooths out transient fluctuations like OS jitter, yielding a stable prediction of G for the upcoming cycle.

Before injecting the load, the agent parses the tensor metadata to obtain the byte size L . Combined with the network link bandwidth C , it can derive the theoretical transmission time $T = \frac{L}{C}$. This process is completed entirely locally and involves only lightweight constant-time operations, including timestamp updates, a fixed-size sliding-window average, simple arithmetic score computation, and DSCP marking.

B. Priority Calculation and Queue Mapping

The core of PQ design lies in mapping local metrics G and T to network priorities. This mapping is based on the

derivation of scheduling policies. Here, we find a fundamental difference between the two optimization objectives, leading to different design strategies.

Delay Minimization. Our theoretical derivation for minimizing waiting time yields the following condition:

$$T_i(G_i + T_i) < T_j(G_j + T_j) \quad (15)$$

A key observation here is that the local node does not need to know information about other tasks. As proven in the derivation, the relative order of competing flows depends solely on the local G_i and T_i .

This means that for the objective of minimizing delay, local information is sufficient to achieve global optimality. A single agent does not need to know the state of other flows or the switch queue lengths. It only needs to calculate its own blocking cost score:

$$S_i^T = T_i(G_i + T_i) \quad (16)$$

A lower S_i^T value strictly indicates a higher priority. This allows PQ-T to achieve the exact optimal ordering for normalized waiting-time minimization under the single-bottleneck abstraction, while preserving a completely distributed implementation. In general multi-hop deployments, we interpret this rule as the target local ordering principle enforced at congested points, whereas the end-to-end outcome may additionally depend on path diversity and multi-point contention.

Maximizing Link Utilization. For maximizing link utilization, the theoretical condition is:

$$(G_i + T_i + D)(G_i + T_i + D + T_j) < (G_j + T_j + D)(G_j + T_j + D + T_i) \quad (17)$$

Here, the variable D needs to be measured. Fortunately, queuing time can be approximated using RTT.

Secondly, obtaining the real-time T value for each flow requires complex network telemetry or centralized coordination, which introduces unacceptable delays. We face a choice: introduce a complex system to obtain global information, or rely solely on local information. We choose the latter. We define an approximate link utilization fraction:

$$S_i^L = \frac{T_i}{G_i + T_i} \quad (18)$$

This metric represents the ratio of the communication time that a flow occupies the link to the sum of the communication and non-communication phases. A smaller S_i^L value means that the traffic in network usage is relatively sparse. Prioritizing such traffic approximates the shortest remaining processing time scheduling strategy in a cyclic context. Although this heuristic method is mathematically approximate compared to the globally optimal solution π^* , experiments show that it is robust and statistically maximizes maintaining pipeline saturation without blocking computation. This approximation is particularly important in practical settings where multi-hop coupling and retransmission overhead make exact real-time estimation of the effective service process difficult.

Quantization. Given the limited number of hardware priority queues provided by commercial switches, we implemented an explicit queue resource partitioning strategy. We reserve Queue 0 exclusively for ACK acknowledgment packets and

critical signaling control, utilizing a strict priority mechanism to ensure the absolute smooth operation. The remaining 7 queues are all open to carry computation traffic.

To adapt the continuous priority scores S_i^T or S_i^L to these 7 physical queues, the system employs a unified Global Linear Quantization rule. Instead of relying on complex local historical states for dynamic threshold determination, all agents directly map the calculated scores linearly to DSCP values in the IP header based on a predefined global mapping function. This stateless deterministic mapping ensures consistency in the understanding of priority semantics across all nodes in the network, efficiently translating fine-grained computation scores into differentiated network quality of service. The detailed score calculation and mapping logic are detailed in Lines 6–12 of Algorithm 1.

This result highlights a key distinction between PQ and prior priority-based datacenter designs. Existing approaches typically assign priorities based on generic flow-level attributes or rely on centralized or global coordination to approximate desirable orders. In contrast, our derivation shows that, for the delay-minimization objective in multi-tenant DNN training, the globally optimal order is directly recoverable from purely local task information. Therefore, the role of the PQ agent is not to heuristically guess urgency, but to implement an objective-driven optimal rule in a fully distributed manner.

C. Collaborative Execution

By mapping priority scores to specific DSCP values, PQ performs scheduling through a dual-mode mechanism acting on both the network infrastructure and the host transport layer as shown in the module of Fig. 5 and corresponding to the execution phase in Algorithm 1 (Lines 13–20).

The agent enforces the calculated priorities by modifying the DSCP field in the IP header. In the network core, general-purpose switches configured with Strict Priority Queuing ensure that queue 0 (reserved for control signals) is always processed first. The remaining queues 1 through 7 process data traffic strictly according to their priority order. This hardware-level mechanism creates a physical fast lane, guaranteeing that the urgent traffic we logically identify is forwarded immediately.

However, relying solely on switch priority queues often leads to buffer bloat. Large volumes of low-priority traffic can fill shared buffers, forcing subsequent high-priority bursts to wait for the buffer to clear, or even experience packet loss. To mitigate this problem, PQ implements a complementary proactive ECN intervention mechanism at the transport layer. By differentially configuring the congestion control parameters of RDMA or TCP, the agent makes low-priority flows highly sensitive to ECN signals to proactively yield bandwidth, while high-priority flows are allowed to tolerate slight congestion and bypass slow start, thus building a second line of soft isolation above the physical layer.

High-Priority Aggressive Mode. For traffic mapped to high-priority data queues, the agent instructs the congestion control engine to bypass the traditional slow-start phase to achieve zero round-trip time saturation. This is primarily

Algorithm 1: Collaborative Priority Scheduling

Input: Training flows $\mathcal{F}$; link capacity C ; minimum RTT RTT_{min} ; queue resource $Q_{0...7}$; global quantization function $\mathcal{Q}(\cdot)$; parameters α, β ; parameters α, β

Output: Coordinated transmission with differentiated QoS

```

1 while PQ Agent running on training server do
  // 1. Local Traffic Awareness
2  for each communication primitive in training loop do
3    Intercept  $t_{curr}$  and  $t_{prev}$  to update
      non-communication duration  $G$  via average history:
4     $G \leftarrow \text{AVG}(t_{curr} - t_{prev}, \text{num} = 3)$ 
5    Transmission size  $L$  and time  $T = L/C$ 
  // 2. Priority Calculation and Mapping
6  for each pending flow  $f_i \in \mathcal{F}$  do
7    if Objective == Delay Minimization then
8       $S_i \leftarrow T_i(G_i + T_i)$ ; // PQ-T Score
9    else if Objective == Link Utilization then
10      $S_i \leftarrow T_i/(G_i + T_i)$ ; // PQ-L Score
11    Map score to hardware queue:
       $q_{idx} \leftarrow \mathcal{Q}(S_i), q_{idx} \in \{1 \dots 7\}$ 
12    Mark IP header:  $f_i.DSCP \leftarrow \text{DSCP\_MAP}(q_{idx})$ 
  // 3. Collaborative Execution
13  for each marked flow  $f_i$  do
14    if  $q_{idx}$  is High Priority then
      // High-Priority Aggressive Mode
15    Set initial window  $ICW \leftarrow C \times RTT_{min}$ ;
      // Jump Start
16    Set ECN reduction factor  $\beta \leftarrow 0.9$ ;
      // Accelerated Recovery
17    else
      // Low-Priority Yielding Mode
18    Apply rate clamping  $\lambda \leftarrow \alpha \cdot C$ ; // Pacing
19    Suppress growth slope to 1/4 of standard rate
20    Inject packets into network
  // Network Infrastructure Behavior (SPQ)
21 while Switch operating do
22   Always process  $Q_0$  first
23   Process  $Q_1 \dots Q_7$  according to  $f_i.DSCP$  in strict
      priority order

```

achieved through the jump start mechanism: the agent rewrites the initial congestion window (ICW) to the bandwidth-delay product of the path ($ICW \approx C \times RTT_{min}$). This modification allows the data stream to transmit at full line speed C from the first packet, effectively eliminating ramp-up latency. Furthermore, to maintain high throughput during minor network fluctuations, the system employs an Accelerated Recovery strategy. Upon receiving occasional ECN signals, the engine applies a gentle reduction factor ($\beta = 0.9$), preventing unnecessary throughput degradation.

Low-Priority Yielding Mode. Conversely, for traffic mapped to low-priority queues, the agent executes a yielding strategy to preserve switch buffer space. The proxy configures the pacer to perform rate clamping, limiting the maximum injection rate to a certain proportion of the link capacity ($\lambda_{pacing} = \alpha \cdot C$, where $\alpha < 1.0$), thereby artificially creating bandwidth redundancy. Additionally, the system enforces conservative growth by suppressing the congestion control growth function. Even without receiving congestion signals,

the congestion window increases linearly at a reduced slope (1/4 of the standard rate). This prevents large, low-priority flows from aggressively probing bandwidth, thus reserving critical buffer space for potential high-priority burst traffic.

By actively limiting low-priority traffic at the source, PQ ensures that network buffers remain at low levels, guaranteeing that incoming high-priority tasks experience virtually no queuing delay without the need for centralized admission control.

This coordinated execution mechanism ensures that the optimal ordering computed by the logical modules is faithfully executed by the physical network, enabling communication interleaving without centralized coordination.

D. Computational Overhead and Scalability

We analyze the host-side overhead of PQ from the perspective of computational complexity. PQ is fully distributed: each training server runs an autonomous local agent and relies only on locally observable metrics, without requiring global state synchronization or centralized coordination. Therefore, the overhead on one host is independent of the cluster size.

For each communication primitive, the local bookkeeping cost is $O(1)$, including timestamp interception, updating G via a fixed-size sliding-window average, parsing tensor metadata to derive T , computing a priority score, and marking the DSCP field. The only non-constant component is the per-round scan over the pending local flows. If a host has F_{local} pending flows, the priority calculation and queue mapping cost is $O(F_{local})$ for that host.

This complexity is lightweight in practice for two reasons. First, PQ does not maintain cluster-wide state, so scaling the cluster to thousands of devices does not increase the per-host control complexity. Second, PQ avoids per-packet decision making in the host control path: once the score is computed and the DSCP field is marked, the fine-grained enforcement is offloaded to commodity switch priority queues. Hence, PQ's control path scales with local flow concurrency rather than global cluster size and does not introduce a centralized CPU bottleneck.

VI. EVALUATION

A. Experimental Setup

Simulation Network. To evaluate the multi-tenant scheduling performance of PQ, we constructed a discrete-time flow simulator in Python, comprising approximately 4,000 lines of code. The simulator is intended as a mechanism-faithful model of PQ's network-side control loop, explicitly capturing topology, finite buffers, queueing, ECN/congestion feedback, sender-side AIMD, and asynchronous (G, T) task lifecycles, rather than as a cycle-accurate emulator of GPU or NIC microarchitecture. The simulated network employs a hybrid topology consisting of 2 spine switches and 32 Top-of-Rack (ToR) switches. These switches connect a total of 192 hosts, with each ToR connecting six hosts. The network is configured with an oversubscription ratio of 3:1 to simulate realistic congestion scenarios. All network links operate at 60 Gbps, and the propagation delay between ToRs and spine switches

is set to 0.001 ms. The switch queue capacity is set to 500 KB per port to reflect the buffer constraints of switches.

Task Types and Workloads. The experiments utilize eight representative models: GPT-2 [41], GPT-3 [7], ResNet50 [17], VGG16, VGG19 [46], BERT [10], RoBERTa [27], and CamemBERT [31]. These models exhibit distinct communication patterns, representing the diverse challenges in multi-tenant scheduling. We evaluated two different scenarios. One is a homogeneous scenario, where six types of tasks with similar transmission requirements are randomly deployed on a multi-tenant cluster, with one task deployed per server (excluding GPT-3); the other is a heterogeneous scenario, where tasks with significantly different transmission requirements coexist, with a 1:3 ratio of GPT-3 to other types of deployments, also with one task deployed per server. The experimental deployment conditions approximate theoretical limits. To evaluate bandwidth contention, we define an intensity metric: an intensity of 0 corresponds to the deployment of the four basic tasks in the heterogeneous scenario. For every 0.2 increase in intensity, one additional competing task (excluding GPT-3) is deployed in one ToR.

Performance Metrics. We evaluate performance using the following metrics. Iteration time, defined as the interval between two consecutive training iterations; Delay time, representing the queueing time experienced by tasks while waiting for resources. Convergence time refers to the time required to reach a stable iteration state. Link utilization presented as a Cumulative Distribution Function (CDF) calculated over a sliding window of $[0, 2]$ seconds with a step of 0.01 seconds.

Baselines. We benchmark against three baselines: FQ, a relatively fair flow control queuing mechanism; MLTCP, an end-host rate control method specifically designed for distributed DNN training to achieve inter-job communication interleaving; and pFabric, a strategy inspired by traditional data center scheduling based on remaining data volume, implementing a shortest remaining time first communication policy. To further analyze the impact of scheduling strategies on system performance, we implemented two variants of the PQ algorithm. PQ-T aims for a minimum normalized relative waiting time strategy, and PQ-L aims for a maximum link utilization strategy.

B. Iteration Time Performance

We conducted a homogeneous task experiment, as shown in Fig. 6a, in which six types of tasks with comparable transmission requirements were randomly assigned on a multi-tenant cluster server. Observing the overall trend, the PQ-T scheme performed best, followed closely by PQ-L, while pFabric and MLTCP occupied the middle tier, and the conventional FQ scheduling algorithm performed the worst. Specifically, PQ-T achieved the lowest average iteration time of 1.92 seconds, representing a 29.2% performance improvement over the least efficient FQ scheme at 2.71 seconds. In comparison, PQ-L had an average time of 1.94 seconds, outperforming pFabric’s 2.22 seconds and MLTCP’s 2.35 seconds. This superior performance stems from the PQ framework’s use of priority queues as a robust buffer to mitigate computational jitter, ensuring

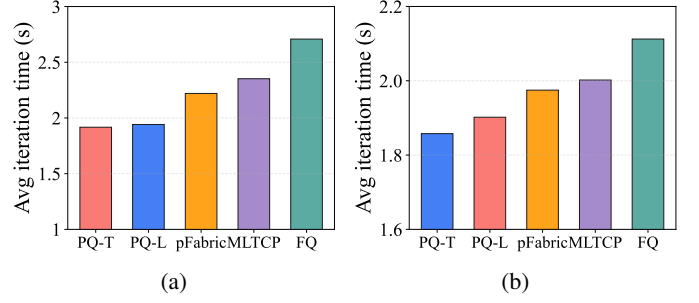


Fig. 6: Average iteration time performance in (a) homogeneous workloads and (b) heterogeneous workloads.

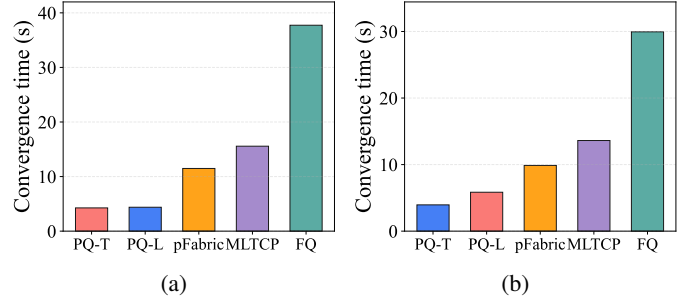


Fig. 7: Convergence time required to reach stability in (a) homogeneous workloads and (b) heterogeneous workloads.

that critical flows experiencing computational blocking are processed immediately while non-critical flows are temporarily buffered. This approach enables efficient communication interleaving without relying on global clock synchronization.

Fig. 6b presents a heterogeneous scenario comprising mixed tasks with significantly different transmission requirements, where the deployment ratio of GPT-3 to other task types was set at 1 to 3. In this complex environment, the performance ranking of the protocols remained consistent with previous observations, with the PQ series methods outperforming pFabric and MLTCP and FQ remaining the least efficient. Specifically, PQ-T achieved a leading iteration time of 1.86 seconds, corresponding to a 12.1% reduction compared with the FQ scheme’s 2.11 seconds. Meanwhile, PQ-L recorded an average time of 1.90 seconds, markedly better than pFabric’s 1.97 seconds and MLTCP’s 2.01 seconds. The experimental results demonstrate that by dynamically mapping task urgency derived from the non-communication to communication ratio to DSCP tags, PQ can leverage hardware queues to sustain extremely high pipeline saturation in heterogeneous workloads, effectively ensuring overall training progress even in the presence of noisy background traffic.

C. Convergence Performance

We evaluated the convergence time required to reach a stable iterative state in a homogeneous task scenario, as shown in Fig. 7a. When deploying task types with similar transmission requirements, the PQ-T and PQ-L schemes exhibited the fastest convergence speeds, while pFabric and MLTCP performed moderately, and the FQ baseline scheme had the

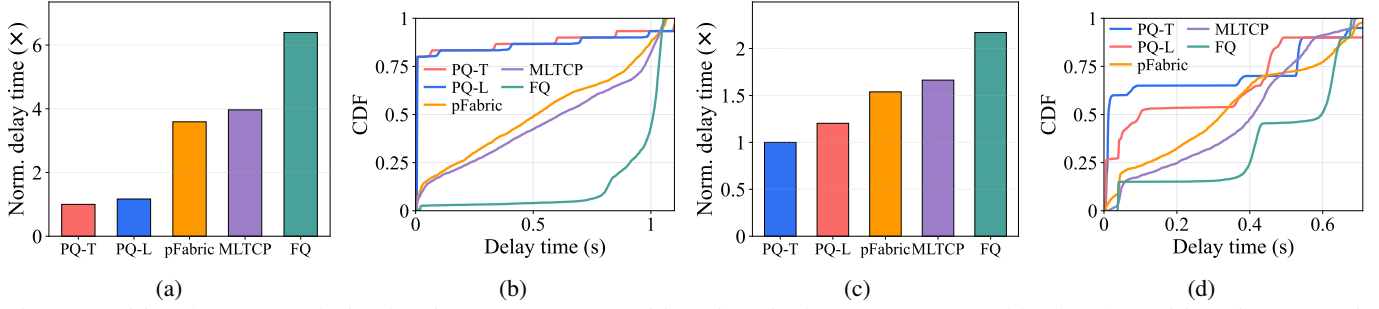


Fig. 8: Waiting latency analysis showing (a) average waiting time in homogeneous workloads, (b) waiting time CDF in homogeneous workloads, (c) average waiting time in heterogeneous workloads, and (d) waiting time CDF in heterogeneous workloads.

longest stabilization time. Specifically, PQ-T reached a stable state in 4.26 seconds, an 8 $\times$ improvement in convergence speed compared to the FQ scheme, which required 37.73 seconds. In contrast, the convergence times for PQ-L, pFabric, and MLTCP were 4.38 seconds, 11.48 seconds, and 15.57 seconds, respectively. This rapid convergence is attributed to the proactive nature of the PQ framework, which utilizes priority queues as powerful buffers capable of immediately absorbing traffic conflicts caused by computational jitter. Unlike passive rate control methods that must wait for congestion signals to adjust windows, PQ's priority-based scheduling mechanism allows the system to transition to a stable interleaved state within just a few cycles.

Fig. 7b shows the convergence performance in a heterogeneous scenario containing a 1:3 mix of GPT-3 and other task types. The performance ranking remained consistent, with PQ variants significantly outperforming the baseline schemes in achieving rapid stability. PQ-T achieved the shortest convergence time of 3.95 seconds, 86.8% faster than the FQ scheme's 29.95 seconds. Meanwhile, PQ-L's convergence time was 5.84 seconds, significantly better than pFabric's 9.87 seconds and MLTCP's 13.62 seconds. The results demonstrate that the stateless deterministic mapping used by PQ eliminates the need for complex parameter tuning or global synchronization. This allows heterogeneous tasks to quickly resolve conflicts and reach an optimal scheduling order even in highly dynamic multi-tenant environments.

D. Delay Time Analysis

We first examine the delay time performance in the homogeneous task scenario in Fig. 8a and Fig. 8b. When six types of tasks with comparable communication requirements are executed in parallel, a distinct performance stratification emerges across different protocols. Using the best-performing PQ-T as the baseline with a normalized delay of 1, the normalized delay time for PQ-L reaches 1.168, while pFabric and MLTCP incur significantly higher delays of 3.591 and 3.967, respectively. The FQ scheme exhibits the worst performance with a normalized value of 6.393. Specifically, PQ-T reduces the normalized delay time by 6.39 times compared to the FQ baseline. The CDF in Fig. 8b further confirms this trend, where the PQ series curves rise sharply at very low delay intervals, while the FQ curve displays a prominent long tail,

indicating that a majority of tasks under FQ suffer from prolonged queuing delays. This advantage stems directly from the PQ-T sorting rule $S_i^T = T_i(G_i + T_i)$, which is theoretically proven to minimize the aggregate normalized waiting time by prioritizing flows that are most likely to block the next cycle.

Fig. 8c and Fig. 8d present the delay time analysis for the heterogeneous scenario. In this mixed-traffic environment, PQ-T maintains its superior performance. With PQ-T as the baseline normalized to 1, the delay times for PQ-L, pFabric, and MLTCP are 1.203, 1.538, and 1.663, respectively, while FQ records a delay of 2.170. Compared to the FQ scheme, PQ-T achieves a 2.17 $\times$ delay time. As observed in Fig. 8d, the delay distribution of the PQ variants is significantly more concentrated and compact than that of the other baselines under heterogeneous workloads. This robustness is attributed to the dynamic mapping mechanism of task urgency to hardware DSCP tags. By identifying the blocking cost of different flows locally and utilizing switch priority queues for isolation, PQ ensures that critical data in heterogeneous tasks can bypass buffer bloat, thereby obtaining a physical-level fast forwarding channel amidst complex background traffic.

E. Link Utilization Performance

In Fig. 9a and Fig. 9b, the link utilization is evaluated across multi-tenant cluster servers with homogeneous tasks. The PQ-L scheme achieves the highest global utilization rate, followed closely by PQ-T, while pFabric, MLTCP, and FQ show significantly lower efficiency. Specifically, with PQ-L normalized to 1, PQ-T maintains a high utilization of 0.983, whereas pFabric, MLTCP, and FQ record values of 0.752, 0.774, and 0.771, respectively. Compared to the FQ baseline, PQ-L achieves a 29.7% improvement in normalized link utilization. The CDF plot in Fig. 9b further reveals that the PQ variants maintain consistently higher utilization rates across the sliding measurement window. By prioritizing traffic with relatively sparse network usage, PQ-L maximizes pipeline saturation and maintains link utilization of the theoretical optimum.

Fig. 9c and Fig. 9d depict the link utilization in a heterogeneous scenario. PQ-L remains the leader in performance, followed by PQ-T and MLTCP, while FQ and pFabric lag behind. With PQ-L as the baseline, PQ-T and MLTCP achieve utilization rates of 0.929 and 0.923, respectively, while FQ

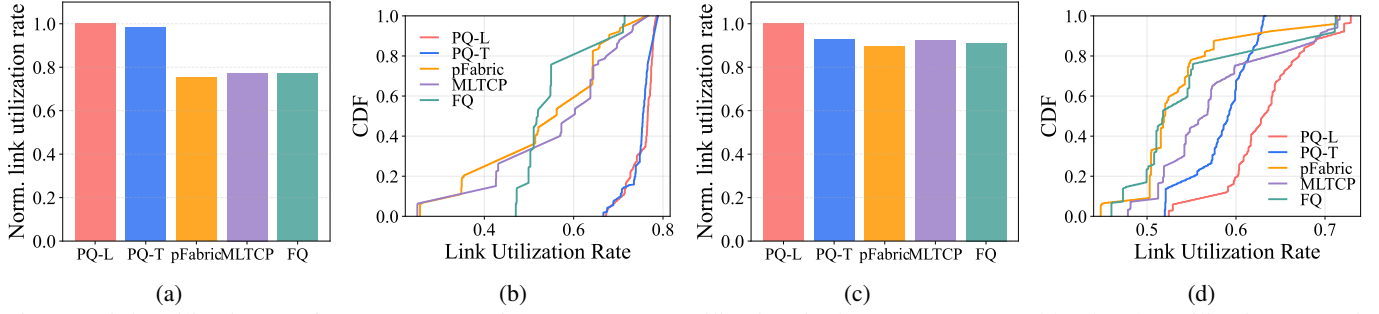


Fig. 9: Link utilization performance presenting (a) average utilization in homogeneous workloads, (b) utilization CDF in homogeneous workloads, (c) average utilization in heterogeneous workloads, and (d) utilization CDF in heterogeneous workloads.

and pFabric are recorded at 0.911 and 0.897. PQ-L provides a 9.8% increase in utilization compared to the FQ scheme in this complex environment. The CDF analysis in Fig. 9d shows that the PQ framework effectively stabilizes link usage even under diverse communication patterns. This robustness is enabled by the collaborative execution mechanism, where high-priority flows bypass traditional slow-start phases using the “jump start” mechanism to achieve zero-RTT saturation. This ensures that network buffers are utilized effectively to sustain training progress despite the complexity of heterogeneous workloads.

F. Performance Scalability under High Load

Contention Intensity and Delay. We evaluate the scalability of different protocols by increasing the contention intensity from 0.2 to 1.0, as shown in Fig. 10a. In this heterogeneous scenario, each 0.2 increase in intensity represents an additional competing task deployed on each server under a ToR switch. As expected, the average delay time for all protocols increases as network contention becomes more severe. From a macro perspective, the PQ variants consistently occupy the leftmost position in the CDF plots across all intensity levels, signifying the lowest delay, while the FQ baseline remains the rightmost with the highest latency. Specifically, at a high intensity of 1.0, PQ-T achieves a mean delay time of 0.338 seconds, whereas pFabric, MLTCP, and FQ record mean delays of 0.38 seconds, 0.419 seconds, and 0.468 seconds, respectively. At this maximum load, PQ-T reduces the average delay time by 27.8% compared to the FQ baseline.

Stability and Jitter Analysis. Fig. 10b presents the scatter plot distribution of delay times across the varying contention intensities. The PQ series exhibits significantly tighter clusters compared to the widely dispersed distributions of pFabric, MLTCP, and FQ, particularly as intensity increases. For instance, at intensity 0.2, PQ-T maintains a mean delay of 0.182 seconds ($1.76\times$ faster than FQ), and even at intensity 1.0, its distribution remains relatively stable compared to the baseline. This robustness under high load is a direct result of the priority queuing mechanism, which allows switches to act as elastic buffers. By prioritizing critical flows that would otherwise block GPU computation, PQ absorbs stochastic computation jitter and maintains high pipeline saturation. Furthermore, the collaborative execution’s proactive ECN intervention ensures that low-priority traffic yields bandwidth, effectively reserving

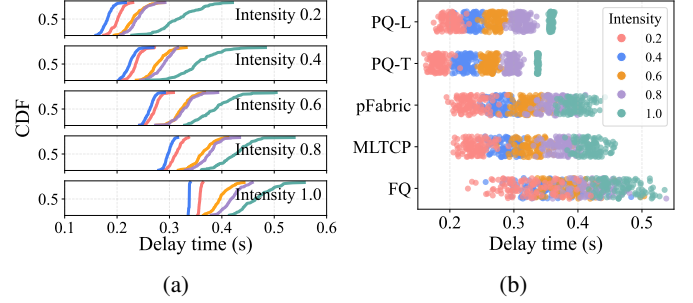


Fig. 10: Performance scalability under high load showing (a) CDF comparison of different protocols and (b) scatter plot distribution of delay times across increasing intensity.

buffer space for high-priority bursts without the need for centralized coordination.

G. Trade-off Analysis: PQ-T vs. PQ-L

Finally, we analyze the performance trade-offs between the two PQ variants to verify if they align with their respective theoretical design goals. As shown in Fig. 8, PQ-T consistently achieves the lowest waiting latency across both scenarios. In the homogeneous case illustrated in Fig. 8a, PQ-T maintains a 14.4% lower normalized delay compared to PQ-L. This gap is further evidenced in the heterogeneous scenario presented in Fig. 8c, where PQ-T yields the most compact delay distribution. These results validate that the S_i^T strategy effectively minimizes the time GPUs spend idling for communication by prioritizing flows with higher blocking costs to the cycle.

Conversely, regarding network efficiency, PQ-L demonstrates its superiority in maximizing bandwidth exploitation. As illustrated in Fig. 9, PQ-L achieves the highest global link utilization, outperforming PQ-T by approximately 1.7% in homogeneous workloads and 7.1% in heterogeneous workloads as shown in Fig. 9c. While PQ-T already significantly outperforms traditional baselines like FQ and pFabric, the strategy of PQ-L to prioritize flows with relatively sparse network usage allows it to fill the gaps in the pipeline more aggressively. This comparison confirms that the PQ framework provides a programmable abstraction where operators can seamlessly pivot the system focus between minimizing task-level latency

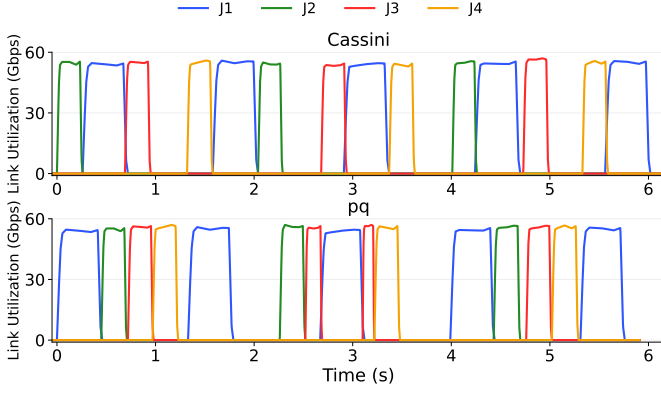


Fig. 11: Comparison between a CASSINI-like placement policy and PQ.

and maximizing cluster-wide throughput by simply swapping the mapping logic between task urgency and DSCP tags.

H. Targeted Comparison with Matching-Based Scheduling

Since CASSINI is explicitly discussed in the Introduction as a representative scheduler based on pre-matched communication placement, we add a targeted comparison here to directly validate that criticism. Our goal is not to argue against CASSINI as a whole, but to examine the limitation of the matching-based scheduling principle it represents. Therefore, instead of re-implementing the complete CASSINI framework, we construct a focused experiment that captures its core mechanism: pre-matched communication windows based on placement and alignment.

Fig. 11 shows the temporal link-utilization traces under a CASSINI-like matching-based policy and under PQ in heterogeneous scenario. The key observation is that when the pre-matched communication placement is imperfect, visible idle gaps appear because the link must wait for the designated matched flow to arrive. In contrast, PQ operates at the flow level and remains work-conserving: whenever ready traffic exists, the bottleneck link continues forwarding without leaving such matching-induced gaps. This result directly supports our motivation that the weakness lies in rigid matching-based placement under dynamic conditions, rather than in any particular implementation detail of CASSINI itself.

I. Sensitivity to α and β

PQ relies on two key control parameters in the collaborative execution stage: the rate-clamping coefficient α and the multiplicative decrease factor β . To evaluate whether PQ is sensitive to these parameters, we conducted an additional parameter-sensitivity study using the same discrete-time simulator framework as in the main evaluation, while focusing on a controlled homogeneous workload setting to isolate the effect of parameter variation.

In Fig. 12, we used the same representative model family as in the main evaluation and ran the simulator under the same periodic asynchronous task abstraction. We swept $\alpha \in \{0, 0.25, 0.5, 0.75, 1.0\}$ and $\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$.

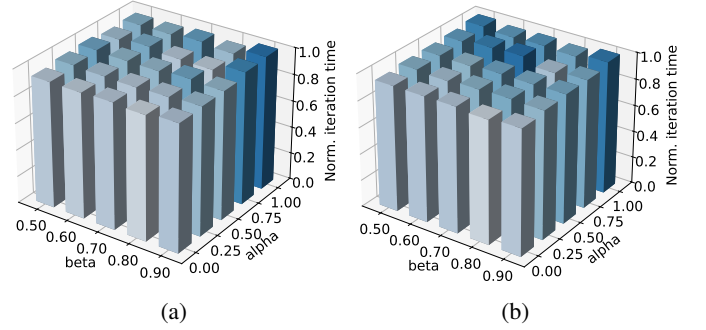


Fig. 12: Sensitivity analysis under different parameter settings: (a) PQ-L and (b) PQ-T.

For each parameter pair, we recorded the average iteration time and normalized it by the maximum average iteration time within the same sensitivity surface. Fig. X and Fig. Y show the resulting normalized average iteration time for PQ-T and PQ-L, respectively.

For both PQ-T and PQ-L, the normalized performance surfaces remain smooth over the entire parameter range, without sharp performance cliffs. This indicates that PQ does not rely on fragile fine-grained parameter tuning. More importantly, even when moving from the best-performing region to the most fairness-preserving region, the maximum relative performance degradation remains below 10%, which suggests that preserving more short-term fairness only incurs a limited efficiency penalty.

We further observe a clear monotonic trend in both variants: smaller α and smaller β generally lead to better normalized iteration time. Intuitively, reducing α strengthens the suppression of low-priority traffic, while reducing β makes rate reduction more aggressive once congestion feedback is observed. As a result, both parameters improve prioritization efficiency when decreased. However, their impacts are not equally strong. The variation along the α dimension is visibly larger than that along the β dimension, indicating that α is the dominant sensitivity knob, whereas β has a comparatively milder second-order effect.

For PQ-T, the best relative performance appears in the small- α region, while a broad middle range still maintains near-best normalized iteration time. PQ-L shows a similar pattern, with relatively smooth changes across β and a somewhat stronger response to α . Overall, these results indicate that PQ is robust across a broad range of practical parameter settings: although more aggressive configurations can further improve relative iteration efficiency, a wide parameter region already provides stable near-optimal performance.

VII. DISCUSSION

In this section, we discuss the evolution path and design limitations of the PQ framework in real-world production environments, aiming to provide a reference for future in-depth research in this field.

Trade-off between Fully Distributed Architecture and Globally Optimal Strategy. PQ adopts a fully distributed design in which each training server independently determines

priorities from locally measured non-communication time G and communication demand T , thereby avoiding the delay and coordination overhead of centralized state synchronization. Our theoretical analysis is developed under a single effective bottleneck abstraction, where such local decisions can expose objective-driven ordering rules in a tractable manner. In practice, however, deployment occurs over multi-hop spine-leaf networks with oversubscription, switch queues, and distributed contention. As a result, when multiple comparable bottlenecks are simultaneously active, or when ECMP path diversity causes different flows to experience distinct congestion locations, the exact global-optimality guarantee of the analytical model no longer directly applies. In these cases, PQ should be interpreted as a distributed scheduler guided by the dominant-bottleneck model rather than a universal exact solution for arbitrary topologies. A promising future direction is to introduce lightweight global context, such as coarse-grained load summaries from switches or weakly coupled controllers, so that local agents can adapt their quantization and priority mapping more effectively while preserving the efficiency of distributed execution.

Scheduling Conflicts Caused by Limited Physical Priority Queue Resources. The scheduling performance of PQ highly depends on the hardware priority queues provided by commercial switches. In this paper, we used 7 queues for priority quantization in our experiments. However, in large-scale multi-tenant AI clusters, the number of concurrently running training tasks may far exceed 7 models. When the priority scores of a large number of tasks are mapped to the same physical queue, the fine-grained sorting capability advocated by PQ will degenerate into traditional first-in, first-out or fair queuing modes within that queue. This quantization collision is particularly prominent in scenarios where tasks are highly homogeneous and the concurrency scale is extremely large. How to maintain scheduling sensitivity under limited hardware queue resources through dynamic weight allocation or queue remapping mechanisms is an engineering challenge that must be addressed when deploying this solution in more complex public cloud environments in the future.

Balancing Controlled Unfairness and Cluster Productivity. The core of the PQ framework lies in actively introducing local traffic unfairness, that is, ensuring zero-latency forwarding of critical path traffic by having non-critical path traffic actively yield. This controlled unfairness might be considered a flaw in traditional cloud services, but in DNN training scenarios, it is a crucial means of improving overall cluster productivity. This is because traditional bandwidth fairness often leads to expensive GPU resources becoming inefficiently idle while waiting for network synchronization. Although experiments have shown that PQ can significantly reduce average iteration time, in a multi-tenant environment, explaining the uneven bandwidth allocation to different tenants and ensuring that even low-priority traffic does not time out due to prolonged queuing is a complex balancing act involving resource billing and SLA guarantees. This shift in perspective from network fairness to task progress fairness provides important insights for designing future AI-oriented data center network architectures.

PQ is intentionally designed around task-progress fairness rather than strict per-tenant bandwidth fairness. In the current design, low-priority flows are not denied service; instead, they are rate-clamped and made more sensitive to congestion signals so that they yield bandwidth during bursts of critical traffic. Moreover, priorities are recomputed repeatedly for each communication phase based on the locally observed non-communication and communication metrics. Given the iterative structure of DNN training, this dynamic reprioritization reduces the risk of persistent starvation in typical workloads, since a flow that is non-critical in one phase may become urgent in a later phase. However, PQ does not currently provide a formal starvation-free guarantee or a strict per-tenant isolation guarantee under persistent overload or pathological priority skew. Incorporating tenant-aware minimum-share control, priority aging, or hybrid priority/fair-queuing mechanisms is an important direction for future work.

VIII. CONCLUSION

Inspired by the structured and interdependent characteristics of traffic trained by deep neural networks, PQ proposes a rule-based distributed traffic control method that bridges the gap between local unfairness and global scheduling efficiency. This framework abandons fragile global synchronization mechanisms in favor of hardware-compatible tagging and queuing mechanisms, utilizing standard priority queues in commercial switches as elastic buffers to cope with random computational jitter. Our design achieves robust communication interleaving without requiring non-standard hardware modifications. Ultimately, PQ provides a perspective for rethinking traffic control in AI clusters, and we hope this work will inspire further exploration of co-designed traffic control and scheduling primitives in future multi-tenant training scenarios.

REFERENCES

- [1] Mohammad Al-Fares et al. Hedera: dynamic flow scheduling for data center networks. In *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation*, NSDI'10, page 19, USA, 2010. USENIX Association.
- [2] Mohammad Alizadeh et al. pfabric: minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 435–446, New York, NY, USA, 2013. ACM.
- [3] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 63–74, New York, NY, USA, 2010. ACM.
- [4] Serhat Arslan et al. Green With Envy: Unfair Congestion Control Algorithms Can Be More Energy Efficient. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, HotNets '23, 2023.
- [5] Andrea Baiocchi et al. Yeah-tcp: Yet another highspeed tcp.
- [6] L.S. Brakmo et al. Tcp vegas: end to end congestion avoidance on a global internet. *IEEE Journal on Selected Areas in Communications*, 13(8):1465–1480, 1995.
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [8] Mosharaf Chowdhury et al. Efficient coflow scheduling with varys. *SIGCOMM Comput. Commun. Rev.*, 44(4):443–454, aug 2014.
- [9] Mosharaf Chowdhury et al. Efficient coflow scheduling without prior knowledge. *SIGCOMM Comput. Commun. Rev.*, 45(4):393–406, aug 2015.

- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [11] Nandita Dukkupati et al. Processor sharing flows in the internet. In Hermann de Meer and Nina Bhatti, editors, *Quality of Service – IWQoS 2005*, pages 271–285, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [12] M. Gerla et al. Tcp westwood: congestion window control using bandwidth estimation. In *GLOBECOM’01. IEEE Global Telecommunications Conference (Cat. No.01CH37270)*, volume 3, pages 1698–1702 vol.3, 2001.
- [13] Juncheng Gu et al. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, Boston, MA, February 2019. USENIX Association.
- [14] Sangtae Ha et al. Cubic: a new tcp-friendly high-speed tcp variant. *SIGOPS Oper. Syst. Rev.*, 42(5):64–74, jul 2008.
- [15] Mark Handley et al. Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication, SIGCOMM ’17*, page 29–42, New York, NY, USA, 2017. ACM.
- [16] Sayed Hadi Hashemi et al. Tictac: Accelerating distributed deep learning with communication scheduling. In A. Talwalkar, V. Smith, and M. Zaharia, editors, *Proceedings of Machine Learning and Systems*, volume 1, pages 418–430, 2019.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [18] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, et al. Characterization of large language model development in the datacenter. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 709–729, 2024.
- [19] V. Jacobson. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols*, SIGCOMM ’88, page 314–329, New York, NY, USA, 1988. ACM.
- [20] Akshay Jajoo et al. Your coflow has many flows: Sampling them for fun and speed. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 833–848, Renton, WA, July 2019. USENIX Association.
- [21] Suhas Jayaram Subramanya et al. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, page 642–657, New York, NY, USA, 2023. ACM.
- [22] Yimin Jiang et al. A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 463–479. USENIX Association, November 2020.
- [23] Dina Katabi et al. Congestion control for high bandwidth-delay product networks. *SIGCOMM Comput. Commun. Rev.*, 32(4):89–102, aug 2002.
- [24] Gautam Kumar et al. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM ’20, page 514–528, New York, NY, USA, 2020. ACM.
- [25] Jialong Li, Shreyansh Tripathi, Lakshay Rastogi, Yiming Lei, Rui Pan, and Yiting Xia. Optimizing mixture-of-experts inference time combining model deployment and communication scheduling. *arXiv preprint arXiv:2410.17043*, 2024.
- [26] Wenxue Li, Xiangzhou Liu, Yuxuan Li, Yilun Jin, Han Tian, Zhizhen Zhong, Guyue Liu, Ying Zhang, and Kai Chen. Understanding communication characteristics of distributed training. In *Proceedings of the 8th Asia-Pacific Workshop on Networking*, pages 1–8, 2024.
- [27] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [28] Kshiteej Mahajan et al. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, Santa Clara, CA, February 2020. USENIX Association.
- [29] Kshiteej Mahajan et al. Better together: Jointly optimizing ML collective scheduling and execution planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, Boston, MA, April 2023. USENIX Association.
- [30] Luo Mai et al. Optimizing network performance in distributed machine learning. In *7th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 15)*, Santa Clara, CA, July 2015. USENIX Association.
- [31] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric Villemonte de La Clergerie, Djamel Seddah, and Benoît Sagot. Camembert: a tasty french language model. *arXiv preprint arXiv:1911.03894*, 2019.
- [32] William M Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C Snoeren, and George Porter. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1–18, 2020.
- [33] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond {GPUs} for {DNN} scheduling on {Multi-Tenant} clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 579–596, 2022.
- [34] Behnam Montazeri et al. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM ’18*, page 221–235, New York, NY, USA, 2018. ACM.
- [35] Rui Pan et al. Efficient flow scheduling in distributed deep learning training with echelon formation. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks, HotNets ’22*, page 93–100, New York, NY, USA, 2022. ACM.
- [36] Rui Pan, Yiming Lei, Jialong Li, Zhiqiang Xie, Binhang Yuan, and Yiting Xia. Efficient flow scheduling in distributed deep learning training with echelon formation. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks*, pages 93–100, 2022.
- [37] Yanghua Peng et al. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference, EuroSys ’18*, New York, NY, USA, 2018. ACM.
- [38] Yanghua Peng et al. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP ’19*, page 16–29, New York, NY, USA, 2019. ACM.
- [39] Jonathan Perry et al. Fastpass: a centralized “zero-queue” datacenter network. In *Proceedings of the 2014 ACM Conference on SIGCOMM, SIGCOMM ’14*, page 307–318, New York, NY, USA, 2014. ACM.
- [40] Aurick Qiao et al. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 1–18. USENIX Association, July 2021.
- [41] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [42] Sudarsanan Rajasekaran et al. Mltcp: Congestion control for dnn training, 2024.
- [43] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. CASSINI: Network-Aware job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1403–1420, Santa Clara, CA, April 2024. USENIX Association.
- [44] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, et al. Inside the social network’s (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 123–137, 2015.
- [45] Aashaka Shah et al. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, Boston, MA, April 2023. USENIX Association.
- [46] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [47] Abeda Sultana, Fei Xu, Xu Yuan, Li Chen, and Nian-Feng Tzeng. Hadar: Heterogeneity-aware optimization-based online scheduling for deep learning cluster. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 681–691. IEEE, 2024.
- [48] Parvin Taheri et al. Rocc: robust congestion control for rdma. In *Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT ’20*, page 17–30, New York, NY, USA, 2020. ACM.
- [49] Gao, Yunqi and Hu, Bing and Mashhadi, Mahdi Boloursaz and Jin, A-Long and Zhang, Yanfeng and Xiao, Pei and Tafazoli, Rahim and Debbah, Merouane. FlowMoE: A Scalable Pipeline Scheduling Framework for Distributed Mixture-of-Experts Training. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

- [50] Gao, Yunqi and Hu, Bing and Mashhadi, Mahdi Boloursaz and Wang, Wei and Tafazolli, Rahim and Debbah, Merouane. A dynamic sliding window based tensor communication scheduling framework for distributed deep learning. *IEEE Transactions on Network Science and Engineering*, 12(2):1080–1095, 2024.
- [51] Balajee Vamanan et al. Deadline-aware datacenter tcp (d2tcp). In *Proceedings of the ACM SIGCOMM 2012 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '12, page 115–126, New York, NY, USA, 2012. ACM.
- [52] Wencong Xiao et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, Carlsbad, CA, October 2018. USENIX Association.
- [53] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. {AntMan}: Dynamic scaling on {GPU} clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 533–548, 2020.
- [54] Lisong Xu et al. Binary increase congestion control (bic) for fast long-distance networks. In *IEEE INFOCOM 2004*, volume 4, pages 2514–2524 vol.4, 2004.
- [55] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmelegy, Scott Shenker, and Ion Stoica. Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European conference on Computer systems*, pages 265–278, 2010.
- [56] Adrian Zaplatel and Fernando Kuipers. Slowdown as a Metric for Congestion Control Fairness. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, HotNets '23, 2023.
- [57] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference*, pages 78–85, 2017.
- [58] Yihao Zhao et al. Multi-resource interleaving for deep learning training. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 428–440, New York, NY, USA, 2022. ACM.
- [59] Pengfei Zheng et al. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 703–723, Boston, MA, April 2023.
- [60] Yibo Zhu et al. Congestion control for large-scale rdma deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, page 523–536, New York, NY, USA, 2015. ACM.